

DIY Challenge Robot Mapping & Localization Workflow Guide

LIO-SAM Prior-Map Generation • FAST-LIO2 Runtime Odometry
EKF Fusion • GPS Integration • Nav2 Bring-Up

ROS 2 Humble • Jetson JetPack 5.x • Hesai QT64

Table of Contents

Section	Topic
1	Architecture Overview
2	Pre-requisites & Environment Setup
3	Mapping Run (LIO-SAM)
4	Saving & Validating the Map
5	Runtime Localization (FAST-LIO2 + EKF)
6	GPS Fusion (navsat_transform + EKF2)
7	Verifying Localization in RViz2
8	Troubleshooting

1 Architecture Overview

The robot uses a two-phase SLAM architecture. During a **mapping session** you drive the full competition area while LIO-SAM builds a globally consistent 3-D point-cloud map using a GTSAM factor graph. At competition time the **runtime localization stack** loads that prior map and runs FAST-LIO2 + dual EKF for real-time pose estimation at 50 Hz.

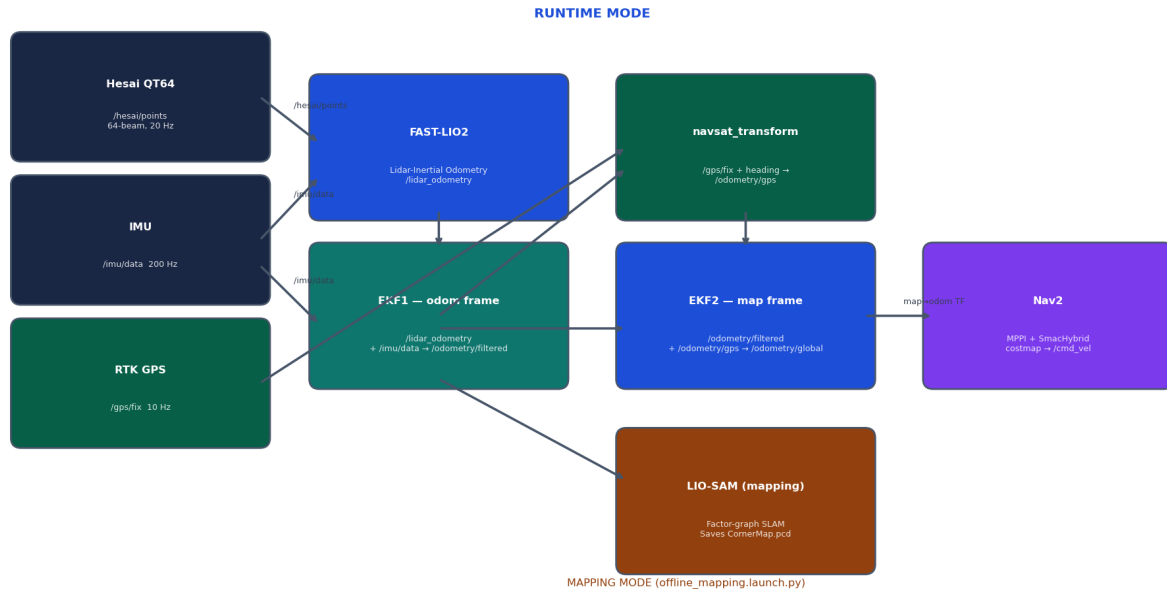


Figure 1 — Full SLAM data-flow. Mapping mode (bottom) is offline; runtime mode (top) runs during competition.

1.1 Mapping Pipeline — LIO-SAM

- **imuPreintegration** — integrates raw /imu/data between lidar scan timestamps to produce high-frequency pose predictions that motion-compensate each sweep.
- **imageProjection** — projects the raw 3-D PointCloud2 into a 2-D range image; applies IMU-derived distortion correction; segments ground plane.
- **featureExtraction** — computes per-point smoothness: low-smoothness points become edge features (walls, poles), high-smoothness become planar features (floor).
- **mapOptimization** — GTSAM factor graph using lidar, IMU, optional GPS, and loop-closure factors. Saves CornerMap.pcd, SurfaceMap.pcd, trajectory.pcd on request.

1.2 Runtime Localization Stack

- **FAST-LIO2** (fastlio_mapping) — lidar-inertial odometry via iKD-Tree; publishes /lidar_odometry at lidar rate.
- **EKF1** (robot_localization, odom frame) — fuses /lidar_odometry + /imu/data at 50 Hz → /odometry/filtered + odom→base_link TF.
- **navsat_transform** — converts /gps/fix lat/lon to /odometry/gps in map frame using EKF1 heading for datum initialization.
- **EKF2** (robot_localization, map frame) — fuses /odometry/filtered + /odometry/gps at 30 Hz → /odometry/global + map→odom TF.

2 Pre-requisites & Environment Setup

2.1 Required Build State

- third_party_ws built and installed: source third_party_ws/install/setup.bash
- DIY workspace built: colcon build --symlink-install inside ros2_ws/
- Calibration files in place (see Calibration Integration Guide)
- Competition area map driven at least once (§3) — or a prior map already in maps/

2.2 Source the Environment

Source Jetson profile (run once per shell session):

```
# On Jetson
source scripts/env.sh jetson

# Verify key variables
echo $DIY_HESAI_IP          # should be 192.168.1.201
echo $DIY_MICRO_ROS_SERIAL # should be /dev/ttyACM0
echo $ROS_DOMAIN_ID        # should be set
```

2.3 Hardware Readiness Checks

Run health_check.sh before any mapping or localization session:

```
bash scripts/health_check.sh
# Checks: node liveness, TF tree completeness, topic rates
# Must show PASS for: hesai/points, imu/data, micro_ros_agent
```

WARNING: Do not proceed to mapping if health_check.sh reports any FAIL. IMU and lidar must both be publishing before starting LIO-SAM.

3 Mapping Run (LIO-SAM)

The mapping launch starts the four LIO-SAM nodes (imuPreintegration → imageProjection → featureExtraction → mapOptimization) plus an optional RViz window so you can monitor map quality in real time.

3.1 Start the Mapping Session

Terminal 1 — start mapping:

```
source scripts/env.sh jetson
ros2 launch diy_localization offline_mapping.launch.py use_rviz:=true
```

Terminal 2 — start hardware drivers (lidar + IMU) separately if not already up:

```
# Hesai lidar driver
ros2 launch hesai_ros_driver manager.launch.py lidar_ip:=$DIY_HESAI_IP
```

```
# IMU (via micro-ROS if STM32 is the IMU gateway)
```

```
ros2 run micro_ros_agent micro_ros_agent serial --dev $DIY_MICRO_ROS_SERIAL -b 921600
```

NOTE: The `offline_mapping.launch.py` file does NOT start hardware drivers. They must be running before or alongside the mapping launch.

3.2 Drive the Mapping Area

- Drive at **0.3–0.5 m/s** maximum — faster motion increases lidar distortion.
- Cover the entire competition area at least once, including all corners.
- Make **at least two loops** so LIO-SAM can form loop-closure factors.
- Keep IMU data continuous — pausing is fine, but do not unplug while mapping.
- Watch RViz: green/blue map cloud should grow steadily without sudden jumps.
- If the map drifts badly, stop, fix calibration, and restart the session.

TIP: For best loop-closure: drive a figure-8 or return to the start point exactly. LIO-SAM triggers ICP-based closure when the robot re-visits a keyframe within the configured radius (default ~6 m in `params.yaml`).

3.3 Monitoring in RViz

- `/lio_sam/mapping/cloud_registered` — accumulated point-cloud map
- `/lio_sam/mapping/trajectory` — robot path markers
- `/lio_sam/mapping/loop_closure` — green lines = accepted loop closures

4 Saving & Validating the Map

4.1 Save the Map via Service Call

Save the map while `mapOptimization` is still running:

```
ros2 service call /lio_sam/save_map \
  lio_sam/srv/SaveMap \
  "{resolution: 0.2, destination: ''}"

# Default output: ~/Documents/lio_sam_directory/
#   CornerMap.pcd    – edge features (walls, poles)
#   SurfaceMap.pcd  – planar features (floor)
#   trajectory.pcd   – robot path
#   global_map.pcd   – merged full map
```

WARNING: Call `/lio_sam/save_map` BEFORE killing the mapping launch. Terminating the process without saving loses the map.

4.2 Copy Map to challenge_bringup

Move map files into the package:

```
# Create a dated directory for backups
MAP_DATE=$(date +%Y%m%d_%H%M)
mkdir -p src/challenge_bringup/maps/map_$MAP_DATE

cp ~/Documents/lio_sam_directory/*.pcd \
  src/challenge_bringup/maps/map_$MAP_DATE/

# Point static_map.yaml at the new map
# Edit maps/static_map.yaml: set pcd_files to the new paths
```

4.3 Update static_map.yaml

`src/challenge_bringup/maps/static_map.yaml`

```
# Update pcd_file to your new map directory
pcd_file: "maps/map_20240601_1430/global_map.pcd"
resolution: 0.2
```

4.4 Validate Map Quality

Quick visual validation with `pcl_viewer` (apt install `pcl-tools`):

```
pcl_viewer ~/Documents/lio_sam_directory/global_map.pcd
# Look for: clear walls, no double-surfaces, no ghost features
```

- Walls and pillars should appear as tight vertical planes (< 5 cm thickness).
- Floor should be a single horizontal plane with no gaps.
- Run laps again if map shows large drift artefacts — check lidar-IMU calibration.

5 Runtime Localization (FAST-LIO2 + EKF)

The runtime localization launch starts FAST-LIO2 in scan-matching mode (using the prior PCD map), EKF1 for odom-frame fusion, and optionally navsat_transform + EKF2 for GPS-anchored global position.

5.1 Launch Localization

Start runtime localization (GPS enabled):

```
source scripts/env.sh jetson
ros2 launch diy_localization localization.launch.py \
  mode:=runtime \
  use_gps:=true \
  config_file:=fast_lio_hesai_qt64.yaml
```

GPS-denied environment (indoor / tunnel):

```
ros2 launch diy_localization localization.launch.py \
  mode:=runtime \
  use_gps:=false
# FAST-LIO2 PGO provides map→odom without GPS
```

5.2 Data Flow in Runtime Mode

Runtime Topic Flow

Step	Publisher	Topic	Consumer
1	Hesai driver	/hesai/points	FAST-LIO2
2	IMU driver	/imu/data	FAST-LIO2, EKF1
3	FAST-LIO2	/lidar_odometry	EKF1
4	EKF1	/odometry/filtered	EKF2, Nav2
5	EKF1	odom→base_link TF	TF tree
6	GPS receiver	/gps/fix	navsat_transform
7	navsat_transform	/odometry/gps	EKF2
8	EKF2	/odometry/global	Nav2
9	EKF2	map→odom TF	TF tree

5.3 FAST-LIO2 Configuration Key Parameters

src/diy_localization/config/fast_lio_hesai_qt64.yaml (excerpt):

```
common:
  lid_topic: /hesai/points
  imu_topic: /imu/data
  time_sync_en: false

preprocess:
  lidar_type: 2          # Velodyne-style PointCloud2
  scan_line: 64          # QT64 line count
  scan_rate: 20          # Hz – match hardware setting
  blind: 0.3             # metres – minimum lidar range

mapping:
```

```
acc_cov: 0.1          # CALIB: replace from IMU calibration
gyr_cov: 0.1          # CALIB: replace from IMU calibration
b_acc_cov: 0.0001
b_gyr_cov: 0.0001
extrinsic_T: [0, 0, 0] # CALIB: lidar→IMU translation (metres)
extrinsic_R: [1,0,0, 0,1,0, 0,0,1] # CALIB: rotation matrix
extrinsic_est_en: true # Set false after calibration converges
```


6 GPS Fusion (navsat_transform + EKF2)

GPS fusion gives the robot absolute global position, preventing long-term map drift over the competition course. It requires a valid heading from EKF1 before it can initialize the UTM datum.

6.1 navsat_transform Configuration

src/diy_localization/config/navsat_transform.yaml:

```
navsat_transform_node:
  ros__parameters:
    delay: 3.0 # seconds to wait for valid heading
    magnetic_declination_radians: 0.0 # CALIB: NOAA value for competition site
    yaw_offset: 1.5707963267948966 # CALIB: robot +X → magnetic North angle
    use_odometry_yaw: false
    broadcast_utm_transform: true # publishes UTM→map TF for RViz
    zero_altitude: true # z=0 in map frame
    publish_filtered_gps: true
    frequency: 10.0
    two_d_mode: true
```

WARNING: magnetic_declination_radians must be set for the competition GPS location. Look up the value at ngdc.noaa.gov/geomag/calculators/magcalc.shtml and enter the competition venue coordinates. A wrong value causes systematic heading error.

6.2 EKF2 (Global Frame) Configuration

src/diy_localization/config/ekf_global.yaml (key settings):

```
ekf_filter_node_map:
  ros__parameters:
    frequency: 30.0
    two_d_mode: true
    world_frame: map # EKF2 publishes map→odom TF

    odom0: /odometry/filtered # EKF1 output – full pose + velocity
    odom0_config: [true, true, false, # x y z
                  false, false, true, # roll pitch yaw
                  true, true, false, # vx vy vz
                  false, false, true, # vroll vpitch vyaw
                  false, false, false]

    odom1: /odometry/gps # GPS-derived position from navsat_transform
    odom1_config: [true, true, false, # x y only – no heading from GPS
                  false, false, false,
                  false, false, false,
                  false, false, false,
                  false, false, false]
```

NOTE: GPS fix quality determines EKF2 accuracy. RTK fixed solution gives ~2 cm, DGNSS gives ~20 cm, single-point GPS gives ~1–3 m. The EKF2 will automatically use the per-fix covariance from /gps/fix.position_covariance to weight the GPS measurement appropriately.

7 Verifying Localization in RViz2

7.1 Launch RViz

Run localization with RViz enabled:

```
ros2 launch diy_localization localization.launch.py use_rviz:=true
# Uses challenge_bringup/rviz/localization.rviz config
```

7.2 Key Topics to Monitor

Topic	Display	What Good Looks Like
/hesai/points	PointCloud2	Tight point cluster around obstacles, no smear
/tf (TF tree)	TF / Robot Model	All frames: map→odom→base_link→lidar/imu/camera
/odometry/filtered	Odometry	Smooth path, no jumps; covariance ellipse fits robot
/odometry/gps	Odometry	GPS odometry aligns with lidar odometry (< 0.5 m offset)
/odometry/global	Odometry	Globally consistent; minimal divergence from /odometry/filtered
/gps/filtered	NavSatFix → points	GPS track overlaps driven path

7.3 TF Tree Sanity Check

Verify the complete TF chain:

```
ros2 run tf2_tools view_frames
# Expected chain: map → odom → base_link → lidar_link
#                                     → imu_link
#                                     → camera_link
# Any gap means a node failed to start or a topic remapping is wrong.
```

7.4 Quick Sanity Numbers

- **Localization frequency:** /odometry/filtered should publish at ~50 Hz.
- **Pose covariance:** diagonal entries < 0.1 in x/y when FAST-LIO2 is locked.
- **GPS offset:** if /odometry/gps x/y deviates > 2 m from /odometry/filtered in a static test, check yaw_offset in navsat_transform.yaml.
- **IMU data rate:** /imu/data must publish at ≥ 200 Hz for FAST-LIO2.

8 Troubleshooting

Symptom	Likely Cause	Fix
LIO-SAM map drifts / explodes	IMU extrinsic wrong or imu_topic rate too low	Re-run lidar_imu_calib; confirm /imu/data $\geq$ 200 Hz
FAST-LIO2 prints "no point cloud" at startup	/hesai/points not publishing or wrong topic name	Check lid_topic in fast_lio_hesai_qt64.yaml; echo topic
EKF1 covariance grows unbounded	Lidar odometry diverged; no IMU data arriving	Restart FAST-LIO2; check /imu/data; verify ekf_local.yaml
navsat_transform outputs nothing	Valid heading not yet available from EKF1	Wait 3 s for FAST-LIO2 to init (delay: 3.0 in navsat.yaml)
map→odom TF missing	EKF2 not running or use_gps:=false with no fallback	Check ekf_global node status; start with use_gps:=false if no GPS
Nav2 skips goal immediately	map→odom TF stale or base→footprint missing	Check TF times: ros2 run tf2_ros tf2_echo map odom
/gps/fix not publishing	GPS antenna not connected or serial port wrong	Check /dev/ttyUSB* permission; verify nmea_serial_driver params

8.1 Log Inspection Commands

Useful diagnostics:

```
# EKF1 state
ros2 topic echo /odometry/filtered --once

# FAST-LIO2 convergence flag
ros2 topic echo /lidar_odometry --once

# GPS fix quality
ros2 topic echo /gps/fix --once | grep -E "status|covariance"

# TF tree snapshot
ros2 run tf2_tools view_frames && evince frames.pdf

# Record a short debug bag
ros2 bag record -o /tmp/debug_bag \
  /hesai/points /imu/data /odometry/filtered \
  /gps/fix /odometry/gps /tf /tf_static
```

End of Mapping & Localization Workflow Guide. For calibration parameter locations see the Calibration Integration Guide. For Nav2 parameter tuning see the Nav2 Tuning Guide.